

Beta Release Deliverables

Link: https://github.com/Alan-Lee7/SDD_Grocery_Pathfinding

Code Review

Review Received on Grocery Path Finder

Overall, we received feedback that our code was clean, readable, and easy to follow. They said our comments were clear about what each function handled. Along with that, our Factory Pattern was well done, and we had good interface separation. Finally, they said it was good that we had basic bcrypt, JWT, and CORS in place.

One of the main code structures they told us to fix was about `default_matcher` and `store_brand_matcher` because they have almost identical loops where one could just inherit from the other or share a class instead. Another minor thing was that `CouponAwareMatcher` is set up basically as a wrapper around `DefaultMatcher`, which could be merged into it with a flag instead. One SOLID principle pointed out that we needed to improve upon was the Liskov Substitution Principle between the matcher implementations. They also reminded us that we really need to implement unit tests.

A few minor issues that could be resolved include input validation clean-up, making error handling more consistent across the stack, parameterized queries and rate limiting before production, and implementing a linter to keep things consistent.

Review Performed on DJAmber

In our review of DJ Amber's code, we thought that they had a well-structured architecture; their implementation of useAudioEngine is implemented well to subscribe to discrete events, and their implementation of .nvmmrc, .prettierrc, ESLint, and CI linter was all configured.

We recommended that they refactor Timeline.tsx. There is a lot of different features implemented into the same hook that should be separated. We also suggested that they comment their code a little better, especially in some of the .tsx files. Other than that, we thought their code was very well done, and the only other things we could have suggested were just incomplete code that they were still working on.

Review Performed on Team Snake

Overall, we thought that Team Snake's code could use some work. Some of the positives, though, are that the code is organized well, all the backend logic works, and linting systems are set up.

We thought that page.tsx could use a lot of refactoring; the carousel logic, layout, and hardcoded dorm data should all be broken into separate components. Most of the route folders have placeholder content and href='#' links. The hardcoded dorm data should probably be moved to a constant file. All of the img tags are used directly instead of [Next.js](#) Image, to prevent slower loads and layout shift prevention. The useEffect for the infinite loop reset has

images as a dependency, which is not ideal since images are recreated every render. They could have just made the inline textShadow styles as CSS class instead.

Overall, we think that their code has a lot of refactoring to be done, and it seemed that they were not as far along as they should have been so we would assume most of these problems are resolved and implemented into the backend by now, but when we reviewed, they did not really have a backend at all.

Testing Documents

Test Plan

Goals for Testing

There are multiple goals for testing our application, including quality and usability. Regarding quality, the software should be able to determine the shortest path between items within a grocery store and optimize costs between stores. Given a list of groceries by the user, the software looks at the item locations (aisle number) and finds the shortest path between them to reduce the total downtime in a grocery store. This also allows verification of the data parser in the backend as well, ensuring information is correctly being pulled. In terms of optimizing costs, unit tests could be created to verify price comparisons between stores. However, as the database is primarily from one grocery store and other grocery stores are simulated by a multiplier of that original store to show varying prices, testing will have to be done based on coupons being applied. These tests should ensure that the algorithm accurately calculates the total basket price for each store, factoring in discounts. By feeding the system dummy information with varying coupon discounts, it can confirm that the software consistently chooses the cost-effective option.

In terms of usability, the user interface should be able to easily and smoothly navigate the software. The interface should be intuitive, and the user should easily be able to find all the features of the software without a guide. Some features that users will interact with include creating a new account if one doesn't exist, logging into a pre-existing account, creating a shopping list, and finally, the optimization features. Interface testing was already partially done within class, but will be further expanded on with the Alpha/Beta testers.

Testing Strategies

There will primarily be two testing strategies for the software: testing the backend and testing the frontend. The backend testing will focus on the data pipeline and the accuracy of the sorting logic. Unit tests will be implemented to verify that each grocery item is correctly mapped to its designated aisle by comparing the store data and the scraped data. These tests will ensure that the parsing correctly extracts the aisle data and that the sorting algorithm successfully orders the user's grocery list based on this aisle number. By creating various grocery lists, it can be confirmed that the backend consistently produces a sequence of the closest to furthest grocery items. This also ensures information is properly handled across different points of the software, and there are no discrepancies between the backend and frontend.

In terms of frontend testing, it will be tested using unguided User Acceptance Testing (UAT), where users will attempt to navigate the application without any prior knowledge or being provided documentation. This is a black box testing approach, as it will help the team determine how intuitive the frontend is and if adjustments need to be made. To help with this testing, as time is limited, specific tasks will be asked of the user to provide them a general direction of what needs to be completed. By observing the users navigate the application, it will

provide general feedback on possible issues with the interface, and these issues could be resolved after all the testing has been completed.

Resources Used

During the testing, a laptop provided by the team will be given to a user to test the software functionality. They will also be guided by a team member, so they will know what tasks need to be completed. A database of information will also be needed, in this case, the Kroger database, to pull information from for the items and costs. This will be run in the background using the python run time environment and to interact with the frontend, the Chrome web browser will be used. This will also require a form of internet connection as there needs to be a connection made for the API to pull information.

Schedule for Testing

Type of Testing	Date	Description
Unit Testing	Continuous throughout the semester	Testing features as they are developed
Integration Testing	4/3-Now	Testing interactions between systems to see if information is properly handled
Alpha Testing	4/17	Developers test the application for bugs/issues

Type of Testing	Date	Description
Unguided User Acceptance Testing	4/17-4/22	Black box testing to get user feedback

Necessary Setup

Assuming that this is a clean install of the Windows Operating System, the first tasks that need to be completed are installing Node.js, Python, and Git. The versions used are Node v22.13.1 (npm 11.2.0), Python 3.13.5, and Git 2.43.0 (Windows), respectively. Node.js will be necessary to run the frontend of our application, Python to run the backend and data processing, and git to clone the repository. After cloning the repository, certain dependencies need to be installed, which could be done by running “npm i” to install JavaScript dependencies for the frontend, and then going to the backend by doing “cd backend” to install the backend dependencies using “pip install -r requirements.txt” to install all necessary libraries. The tester must also obtain an API key from the Kroger developer portal, which would be placed in the .env file in the root directory. After this is completed, the user could start the backend by doing “python app.py” or “flask run” while in the backend directory and “npm run dev” in the root of the repo to run the frontend. After running the frontend, the tester can control + click on the localhost link in the terminal or copy and paste it into their browser. Another way the application can be run is by using the Make tool. First, Chocolatey needs to be installed before Make can be installed. The version of Chocolatey installed is Version 2.4.3, and the version of Make installed is 4.4.1. After running the backend, in the root directory, you can do “make” to automatically download dependencies and open the browser with the application.

Test Case Specifications:

User Test 1:

Enter valid credentials and verify that the system logs you in, and redirects to the main page.

Sequence of actions:

1. Enter email
2. Enter password
3. Click sign in

User Test 2:

Enter an incorrect password and verify that the system displays an error message and prompts you to try again without logging you in.

Sequence of actions:

1. Enter an email into login page
2. Enter an invalid password
3. Press log in

Verify that the site says invalid password.

User Test 3:

Fill out the registration form with a unique email and strong password, and verify that a new account is created.

Sequence of actions:

1. Click button to sign up

2. Fill out all fields including name, email and password twice
3. Click create an account

Verify no issues with mismatching passwords

4. Attempt to sign in with the same credentials

Verify account was created successfully and signed in

User Test 4:

Attempt to register with an email that already exists, and verify the system tells you why registration failed and prompts you to correct the information.

Sequence of actions:

1. Click sign up
2. Fill out all fields but use email used in test 3

Confirm that notification account already exists

User Test 5:

Add multiple items to the cart, select "Find Optimized Route," choose an entry aisle, and verify the app correctly directs you to the nearest item first and updates the list as each item is marked as grabbed.

Assuming: user is signed in and has selected a store

Sequence of actions:

1. Add any selection of items to your cart
2. Click optimize my list button at the bottom
3. Click the square to check all of the items in the shopping cart

Confirm Items Grabbed pops up and aisle complete and as each item is selected the bar on the top fills up more and more

User Test 6:

Add a single item to the cart, complete the route, and verify the cart shows as all items grabbed and the session ends correctly after the item is grabbed.

1. Add any selection of items to your cart
2. Click optimize my list button at the bottom
3. Click the square to check off each item as you continue putting all items in the shopping cart

Confirm Items Grabbed pops up and aisle complete

User Test 7:

While logged in, enter a valid (Troy) ZIP code and verify the system displays nearby stores, allows you to select one, and refreshes the page with that store's inventory and pricing.

Assuming: user is logged in

Sequence of actions:

1. Type troy zipcode into the field
2. Click search
3. Select your desired store

Confirm you can see a list of items and coupons

User Test 8:

Enter a ZIP code with no nearby stores and verify the system displays an error, keeps the original

store active, and prompts you to try again.

Sequence of actions:

1. Type troy zipcode into the field
2. Click search

Confirm that no stores are found near this zip code

User Test 9:

Add multiple items to the shopping list, click "Optimize My Costs," and verify the system compares prices across nearby stores and suggests a split shopping plan with estimated savings.

assuming : user is logged in and has selected a store

Sequence of actions:

1. Select multiple items and add them to your cart
2. Click optimize my list
3. Then click compare stores
4. Compare what deals other stores have

Confirm that you can see multiple stores

User Test 10:

Navigate to the settings page, select "Large" font size, click apply and save, then navigate to another page and verify the font size change persists.

Assuming: user is logged in and selected a store

Sequence of actions:

1. Click the settings button at the top

2. Select large text button

Confirm that the text is larger on the screen

User Test 11:

Navigate to the settings page, toggle dark mode on, and verify the interface immediately switches to a dark background with light text and the preference is saved across pages.

Assuming: user is logged in and selected a store

Sequence of actions:

1. Click the settings button at the top
2. Select the dark mode button

Confirm that the color scheme is switched to dark mode

User Test 12:

Toggle dark mode on, close the app, reopen it, and verify the dark theme persists from the saved preference without needing to be set again.

Assuming: user is logged in and selected a store

Sequence of actions:

1. Click the settings button at the top
2. Select dark mode button

Confirm the screen is now in dark mode

3. Close the site and reopen it

Confirm that the dark mode is still persisting

Test Results

Template

Below we have a template for the alpha and beta test, where each column is assigned a specific role: text description being a description of what the test cases is about, tester being who the tester are, and completion status being whether or not there are problems or not, finally, the comments explains how well the test went and what sort issues arose.

Test Description	Tester	Completion Status	Comments

Below are the recorded results of the alpha tests done by members of the team. All of the features that were promised have been implemented in one way or another. This test will be a good way of identifying problems with those implementations, the results of this test will determine the team's workload for the final deliverable.

Sebastian

Test Description	Tester	Completion Status	Comments
User Test 1: Valid Login	Sebastian Grammas	Completed	Successfully logged in and show's profile.

Test Description	Tester	Completion Status	Comments
User Test 2: Invalid Login	Sebastian Grammas	Completed	The system displays incorrect credentials and asks users to attempt again.
User Test 3: Valid Registration	Sebastian Grammas	Completed	Was able to create a new account that can be logged into and logged out of.
User Test 4: Invalid Registration	Sebastian Grammas	Completed	The system properly notified that the account already exists and prompted for new credentials.
User Test 5: Optimized Route (Multiple Items)	Sebastian Grammas	Completed	The route generated successfully showed the closest to furthest objects.
User Test 6: Optimized Route (Single Item)	Sebastian Grammas	Completed with Issues	The cart showed all items grabbed, and the session ended correctly.
User Test 7: Store Selection by Valid ZIP	Sebastian Grammas	Completed	Successfully input a zipcode and displayed nearby stores.

Test Description	Tester	Completion Status	Comments
User Test 8: Store Selection by Invalid ZIP	Sebastian Grammas	Completed with Issues	The system displayed an error and prompted the user to try again while keeping the original store active.
User Test 9: Optimize Costs (Compare Stores)	Sebastian Grammas	Completed	The system successfully compared prices across nearby stores
User Test 10: Settings (Large Font)	Sebastian Grammas	Completed	Large font size was applied, saved, and persisted across pages.
User Test 11: Settings (Dark Mode Toggle)	Sebastian Grammas	Completed	Interface successfully switched to dark background, and preference was saved.
User Test 12: Settings (Dark Mode Persistence)	Sebastian Grammas	Completed	The dark theme persisted after closing and reopening the application.

Alan

Test Description	Tester	Completion Status	Comments
User Test 1: Valid Login	Alan Lee	Completed	The user was able to log in successfully and was redirected to the main page. Profile display appeared correctly.
User Test 2: Invalid Login	Alan Lee	Completed	The system rejected invalid credentials
User Test 3: Valid Registration	Alan Lee	Completed	A new account was successful. The system accepted valid credentials and kept account information after registration.
User Test 4: Invalid Registration	Alan Lee	Completed	The system correctly displayed registration failure when an email that has already been used is used again
User Test 5: Optimized Route (Multiple Items)	Alan Lee	Completed	The system generated an optimized route successfully using the mock data. The output displayed is easily readable.
User Test 6: Optimized Route	Alan Lee	Completed with Issues	The cart correctly showed only the singular item and displayed its location

Test Description	Tester	Completion Status	Comments
(Single Item)			
User Test 7: Store Selection by Valid ZIP	Alan Lee	Completed	ZIP code input worked, and store selections displayed properly. Search behavior worked as intended
User Test 8: Store Selection by Invalid ZIP	Alan Lee	Completed with Issues	The system displays an error message and keeps the original store active.
User Test 9: Optimize Costs (Compare Stores)	Alan Lee	Completed	The system evaluated prices across multiple stores and returned the cheapest options.
User Test 10: Settings (Large Font)	Alan Lee	Completed	Font size change was applied and persisted seamlessly across different pages.
User Test 11: Settings (Dark Mode Toggle)	Alan Lee	Completed	The interface switched to a dark background with light text.

Test Description	Tester	Completion Status	Comments
User Test 12: Settings (Dark Mode Persistence)	Alan Lee	Completed	Dark theme persisted from saved preferences upon reopening the application.

Peijing

Test Description	Tester	Completion Status	Comments
User Test 1: Valid Login	Peijing Xu	Completed	The user was able to successfully log in to an already created account without any hiccups.
User Test 2: Invalid Login	Peijing Xu	Completed	The system correctly rejected invalid credentials and prompted the user to try again.
User Test 3: Valid Registration	Peijing Xu	Completed	The user was successfully able to create an account with the sign-up button and was able to log in without any issues with the new email and password.

Test Description	Tester	Completion Status	Comments
User Test 4: Invalid Registration	Peijing Xu	Completed	The user receives a notification confirming the account already exists.
User Test 5: Optimized Route (Multiple Items)	Peijing Xu	Completed	The user clicks on the optimize my list button which displays the aisled based route from the mock store data.
User Test 6: Optimized Route (Single Item)	Peijing Xu	Completed with Issues	The system correctly showed that there was only one item and its aisle
User Test 7: Store Selection by Valid ZIP	Peijing Xu	Completed	The stores show up after inputting the Troy zip code, and the user was able to select one of the displayed stores.
User Test 8: Store Selection by Invalid ZIP	Peijing Xu	Completed with Issues	The system successfully displayed an error about no stores found near the zip code.
User Test 9: Optimize Costs (Compare Stores)	Peijing Xu	Completed	The compare stores feature accurately displayed the price of the shopping cart between two stores.

Test Description	Tester	Completion Status	Comments
User Test 10: Settings (Large Font)	Peijing Xu	Completed	Text is displayed larger on the screen across the pages.
User Test 11: Settings (Dark Mode Toggle)	Peijing Xu	Completed	The color scheme switched to dark mode seamlessly.
User Test 12: Settings (Dark Mode Persistence)	Peijing Xu	Completed	Dark mode continued to persist after reopening the site.

Will

Test Description	Tester	Completion Status	Comments
User Test 1: Valid Login	Qizheng Sui	Completed	Successfully logged in with credentials.
User Test 2: Invalid Login	Qizheng Sui	Completed	The system correctly rejected invalid credentials and prompted the user to try again.

Test Description	Tester	Completion Status	Comments
User Test 3: Valid Registration	Qizheng Sui	Completed	The user can create a new account by inputting their email and password.
User Test 4: Invalid Registration	Qizheng Sui	Completed	The system correctly failed the registration process for a pre-existing email.
User Test 5: Optimized Route (Multiple Items)	Qizheng Sui	Completed	The system generated a route based on the mock data, and the shopping cart is displayed in a readable order.
User Test 6: Optimized Route (Single Item)	Qizheng Sui	Completed with Issues	Completed the route for a single item and closed the session.
User Test 7: Store Selection by Valid ZIP	Qizheng Sui	Completed	Nearby stores shown based on inputted zipcode.
User Test 8: Store Selection by Invalid ZIP	Qizheng Sui	Completed with Issues	The system was able to properly handle an invalid zip code entry.

Test Description	Tester	Completion Status	Comments
User Test 9: Optimize Costs (Compare Stores)	Qizheng Sui	Completed	The program successfully suggested an accurate split shopping plan between locations.
User Test 10: Settings (Large Font)	Qizheng Sui	Completed	The website successfully changed the font size to a larger size across the interface.
User Test 11: Settings (Dark Mode Toggle)	Qizheng Sui	Completed	The user was successfully able to toggle dark mode on.
User Test 12: Settings (Dark Mode Persistence)	Qizheng Sui	Completed	Dark mode preference persisted correctly after reopening.

Project Status Report

The current project can be considered finished, save for some bugs that need to be resolved. From the interim release to the current Beta release, the team spent most of their time debugging the logic and code and attempting to implement item-level filtering. The main issues resolved during this period were the coupon logic, where, when used in conjunction with the compare by store feature, items that have coupons in the second store are not factoring those coupons in, skewing the results and preventing a fair price comparison between stores. This issue

was encountered and solved by Sebastian, who fixed the bug by having the program search through the coupon database and check for possible deals when given the items in the shopping cart. If this bug had not been uncovered, it could have had a devastating effect on the project, as the team is scheduled to present their project next Friday, the 24th. Another impact this bug could've had was an impact on the user's choice between stores; the bug made it so that coupons from the second store did not affect the price of the shopping cart, skewing the user's perception about which store is cheaper. The team also implemented a dark mode feature in the website's settings menu, where users can toggle it on or off. The settings menu is also where the text font can be updated, and luckily, the team did not encounter any issues with this feature as of the Beta release. Finally, the team has begun debugging errors in the code, including features like the item filter, which shows incorrect items when used. This issue was discovered by Sebastian, and the team will continue working on fixing it until the final release. If this bug were not found, it could seriously hurt the image of the product, as the item filter does not work and would leave a negative image of the Grocery Finder in a user's mind. In conclusion, the team is on track for the final deliverables, with only a couple of bugs left to fix before meeting all expected requirements.

Contribution Summary

Sebastian Grammas:

Since the interim release, my job was to implement the dark vs light mode feature that can be toggled in the settings tab for users. I also resolved a few bugs that existed with coupon logic. There were multiple issues, including coupons not being applied when comparing prices across stores, coupons not persisting when switching between stores when comparing prices, and

deals being applied even though the number of items did not meet the “coupon requirement” (4 for 5\$ was still applying the discount for 3 items in cart). I also summarized the code reviews we performed on Team DJ Amber and Team DJ Snake, as well as the code review notes they gave us.

Alan Lee:

Since the interim release, I helped Sebastian with debugging some of the bugs relating to the coupons. I also contributed by helping take notes during the code review session, so it could be summarized for the report and be useful in updating our coding structure. I helped with setting up meeting times and driving the direction of the meeting to meet our goals for our projects. I worked on the test plan portion of the report, helped with coming up with the test cases, and helped format the report.

Peijing Xu:

After the interim release, I contributed to taking notes during the code review session providing additional information on top of Alan and Sebastian’s notes. I also contributed to going over the Testing documents section of this report and wrote the project status portion of this report. I always participated in class during the period between the interim release and the current Beta release and relayed information to teammates who were otherwise preoccupied with work outside of SD&D. I am the one usually asking the clarifying questions that help ease our product’s development.

Qizheng Sui:

My contribution for the Beta release team coordination, team communication, and recording down the test plan. I helped contribute to in class activity and organized the team’s meetings and general reports. Finally, I helped the team to stay organized and on topic.